

APIでデータ取得

*Web*から材料を仕入れる

rkskmt

1

道具から材料へ

2

ここまでで揃った「道具」

- **データを置く** — 変数、コレクション (list / dict / tuple)
- **データを計算する** — NumPy (ベクトル・行列・統計)
- **データを見る** — matplotlib (散布図、折れ線、subplot)

💡 ここからは「材料」

- 道具だけでは何もできない
- **入力データ** (材料) を仕入れる必要がある
- 一番手っ取り早い仕入れ先は **Web**

3

本授業の最終目標

- なんかのデータ群を処理して
- **役に立つ、もしくは興味深い** 出力やサービスのモックを作る
- 今回はその第一歩として **Webからデータを取ってくる** ことをやる

4

Webへアクセス (requests)

5

requests ライブラリ

- PythonでWebにアクセスする **定番ライブラリ**
- 標準ライブラリではないので pip でインストール

```
$ pip install requests
```

Shell

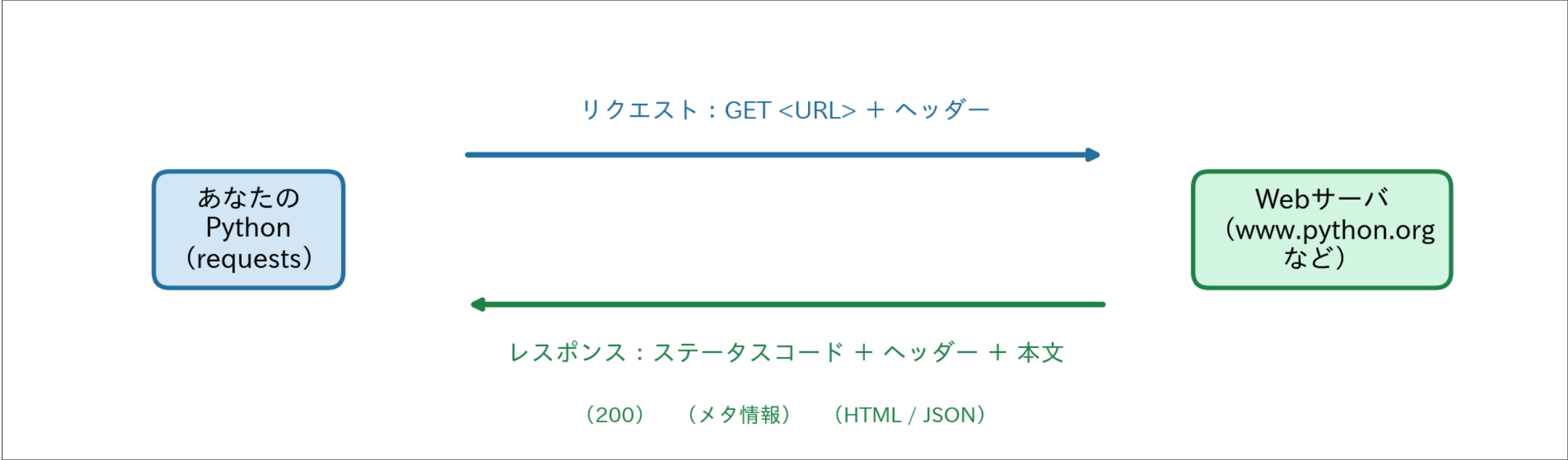
- コード冒頭でいつもどおり import

```
1 import requests
```

Python

6

Webアクセスの正体：リクエストとレスポンス



- Webアクセスは、つねにこの **1往復**
- この往復を **requests.get()** が1行でやってくれる
- 返事は3点セット — **状態（コード）・メタ情報（ヘッダー）・本文** を順に見ていく

7

一番シンプルな使い方

- **requests.get(url)** で **指定URLにアクセス**
- 戻り値の **response** を print すると **状態コード** が見える

```
1 import requests
2
3 response = requests.get('https://www.python.org/')
4 print(response)    # <Response [200]>
```

① 200 とは？

- HTTPの応答コードで **「正常に取得できた」** の意味
- とりあえず **「200か否か」** だけ判断できればOK

8

HTTPステータスコード（普段あなたも見ている）

- ネットを使っていれば **すでに目にしている** 番号たち — その意味を知る

コード	意味	どんなときに出るか（普段の遭遇）	対処
200	OK	正常に表示・取得できた（普段は気づかない）	そのまま処理を続行
403	Forbidden	アクセスを拒否された（権限がない・弾かれた）	権限・ヘッダーを確認
404	Not Found	リンク切れ・URLの打ち間違いで出る定番のページ	URL・記事名を確認
500	Internal Server Error	サイト側が落ちている ・不具合（自分は悪くない）	時間をおいて再試行

- 全部覚える必要なし
- **200番台 = 成功、400番台 = リクエスト側のミス、500番台 = サーバ側のミス** くらいの感覚

❗ 「403」はこのあとすぐ出会う

- 次のWikipedia APIは **User-Agentを付けないと弾かれる** — それがこの403
- 「アクセス拒否」はエラーではなく、**サーバからの正しい返事**

200番台：成功

200 OK

400番台：リクエスト側のミス

403 / 404

500番台：サーバ側のミス

500 / 503

response の中身

- 取得した内容は **プロパティ** で取り出せる

プロパティ	型	説明
.text	str	本文を文字列で取得
.json()	dict	本文がJSONなら dict として取得
.headers	dict風	ヘッダーを辞書風で取得

.text で本文を取得

- HTMLがそのまま str で返ってくる

```
1 import requests
2
3 response = requests.get('https://www.python.org/')
4 print(response.text)    # HTMLの中身
```

Python

11

.headers でメタデータを取得

- ヘッダーは **本文と別についてくる情報**
- 応答時刻、サーバ情報、本文のバイト長、コンテンツ種別 など

```
1 import requests
2
3 response = requests.get('https://www.python.org/')
4 print(response.headers)
```

Python

① 例：ヘッダーの中身

- 全部覚える必要なし
- 「本文の他にもこういう情報をやり取りしてるんだな」くらいでOK

```
1 {
2   "Date": "Mon, 30 Jun 2025 05:00:33 GMT",
3   "Server": "Apache",
4   "Content-Length": "73439",
5   "Content-Type": "text/html",
6   ...
7 }
```

12

Wikipedia の統計APIに触れる

13

Wikipedia Pageviews API

- Wikipedia財団が公開している **ページビュー統計** のAPI
- まずは「2025/6/27 にアクセス数が多かった記事ランキング」を取ってみる

```
1 import requests
2
3 url = "https://wikimedia.org/api/rest_v1/metrics/pageviews/top/ja.wikipedia/all-access/2025/06/27"
4 headers = {'User-Agent': 'example.com/lesson'}
5
6 response = requests.get(url, headers=headers)
7 data = response.json()
8 print(data, type(data))
```

Python

14

ヘッダーを送る理由

- Wikipedia の API は **User-Agent** がないと応答を返さない
- 本来はChromeなどブラウザ名を入れる場所
- 今回は教育目的を示す架空のもの **example.com/lesson** を使う

⚠ ヘッダーなしだと

- アクセス拒否でデータが取れない
- 「公開API」でもヘッダー必須なケースは多い
- API使用前にドキュメントで **必須ヘッダーを確認** する習慣を

15

JSON とは

- **JavaScript Object Notation** の略
- 「ある文法に従って書かれた **ただのテキスト**」
- CSVも仲間（カンマ区切りの構造化テキスト）
- **Webの世界ではデファクトスタンダード**
- まず名前を覚える

```
1 print(response.text) # 中身は生のJSONテキスト
```

Python

```
1 {"items":[{"project":"ja.wikipedia","access":"all-access",
2      "year":"2025","month":"06", ...}]}
```

- **.json()** は **このテキストを dict に変換** するメソッド

JSON : ただのテキスト (str)

```
{
  "items": [
    {
      "project": "ja.wikipedia",
      "access": "all-access",
      "year": "2025",
      ...
    }
  ]
}
```

.json()

dict / list の入れ子（プログラムで扱える）

dict

"items" → list

[0] → dict

```
{
  "project": "ja.wikipedia",
  "access": "all-access"
}
```

16

(余談) 手動で JSON → dict

- 標準ライブラリ **json** で同じことができる
- pip 不要

```
1 import requests
2 import json
3
4 response = requests.get(url, headers=headers)
5 res_dict = json.loads(response.text)
6 print(type(res_dict), res_dict)
7 # <class 'dict'> {'items': [{'project': 'ja.wikipedia', ...}
```

Python

💡 普段は **.json()** でOK

- **requests** が内部でやってくれているだけ
- 「中で何をしているか」を知っておくと **エラー時に強い**

17

APIとエンドポイントの構造

18

API と エンドポイント

- **API** (**A**pplication **P**rogramming **I**nterface)
 - データ取得などのために公開された「窓口」
- **エンドポイント**
 - API の中の、具体的な処理を呼ぶ **個別のURL**
- 例
 - **https://wikimedia.org/api/** 配下が API
 - **/metrics/pageviews/top/...** がエンドポイントのひとつ

19

URLにパラメータが埋め込まれている

- エンドポイントの一部に **値を直接埋め込む** タイプ
- 今回のAPIだと...

```
1 https://wikimedia.org/api/rest_v1/metrics/pageviews/top/  
2     {プロジェクト} / {アクセス方法} / {日付}
```



パラメータ	例の値	意味	他の選択肢
プロジェクト	ja.wikipedia	日本語Wikipedia	en.wikipedia, commons.wikimedia など
アクセス方法	all-access	全アクセス	desktop, mobile- app, mobile-web
日付	2025/06/27	指定日	YYYY/MM/DD の任意 の日付

❗ 覚える必要はない

- API ごとにURL構造はバラバラ
- 使うときに **公式ドキュメントを見る** のが正解
- **Wikimedia REST API ドキュメント** [🔗](#)

個別ページのアクセス推移を取る

別のエンドポイント：per-article

- 特定の記事の、任意の期間のアクセス数（日単位 / 月単位）を返す

```
1 https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/  
2     {project} / {access} / {agent} / {article} / {granularity} / {start} / {end}
```


per-article のパラメータ

パラメータ	選択肢
project	ja.wikipedia など
access	all-access / desktop / mobile-app / mobile-web
agent	all-agents / user / spider / automated
article	任意の記事名（スペースは <u> </u> ）
granularity	daily / monthly
start / end	YYYYMMDD 形式

23

「七夕」のページビューを取得する

- 記事「七夕」の **2025年まるごと1年分** の日別アクセス数を取り、統計量を出す

Python

```
1 import requests
2 import numpy as np
3
4 url = ("https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/"
5       "ja.wikipedia/all-access/user/"
6       "七夕/daily/20250101/20251231")
7 headers = {'User-Agent': 'example.com/lesson'}
8
9 response = requests.get(url, headers=headers)
10 data = response.json()
11
12 # data['items'] は list of dict
13 views = np.array([item['views'] for item in data['items']])
14 dates = np.array([item['timestamp'] for item in data['items']])
15
16 print(f"日数: {len(views)}")
17 print(f"最大: {views.max()}回")
18 print(f"最小: {views.min()}回")
19 print(f"平均: {views.mean():.1f}回")
20 print(f"中央値: {np.median(views):.0f}回")
```

```
1 日数: 365
2 最大: 47656回
3 最小: 58回
4 平均: 467.6回
5 中央値: 154回
```

- 最大が中央値の約300倍**。平均と中央値も大きくズレている
- 可視化編でやった「右に引っ張られた平均」の匂い — **なにかがおかしい**

24

過去の章が一気に効いてくる

- `data['items']` は **list of dict** ([dict章](#) でやった形)
- 内包表記でフィールド抽出 ([list章](#))
- `np.array()` で **ndarray** 化 ([numpy章](#))
- `np.mean` / `np.var` で統計量
- `np.argmax` / `argmin` は **インデックス** を返す (値ではない点に注意)

💡 値もインデックスも両方欲しいとき

- `max_i = np.argmax(views)` で index を得てから
- `views[max_i]` で値、`dates[max_i]` で対応する日付
- 同じ **index** で並ぶ配列同士をリンクさせる のはNumPy/データ処理の頻出パターン

25

まとめ

やりたいこと	コード
アクセス	<code>requests.get(url, headers=...)</code>
状態確認	<code>response.status_code</code> (200 = OK)
本文(str)	<code>response.text</code>
JSON → dict	<code>response.json()</code>
ヘッダー	<code>response.headers</code>
User-Agent	<code>headers={'User-Agent': '...'}</code>
値の集計	<code>np.array([d['views'] for d in items])</code> → <code>mean/var/argmax</code>

- Web から取った dict は今まで通り `for` / `items()` / `np.array()` で処理できる

26

演習

27

演習：desktop と mobile-web の比較



- 同じ期間（2025/01/01 - 2025/12/31）・同じ記事「七夕」で **API を2回呼ぶ**
- **access** を **desktop** と **mobile-web** に切り替える
- それぞれの **最大・最小・平均・分散** を print する
- 結果は捨てずにとっておく — **次回、このデータをグラフで比較する**

💡 ヒント

- 取得結果を **views_desktop** と **views_mobile_web** に分けて入れる
- 同じ **headers** を使い回せる
- 関数化すると重複が消えてスッキリ

28

付録

29

実際のブラウザのUser-Agent

- 普段ブラウザでWebを見るときも **裏でヘッダーが飛んでいる**
- サーバはこの値を見て **PC / スマホ / OS / ブラウザ** を判別している
- レスポンシブやリダイレクトの分岐に使われる

ブラウザ	User-Agent
Chrome (Windows)	Mozilla/5.0 (Windows NT 10.0; Win64; x64) ... Chrome/119.0.0.0 Safari/537.36
Chrome (Mac)	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) ... Chrome/119.0.0.0 Safari/537.36
iOS Safari (iPhone)	Mozilla/5.0 (iPhone; CPU iPhone OS 17_1 like Mac OS X) ... Mobile/15E148 Safari/604.1
iOS Safari (iPad)	Mozilla/5.0 (iPad; CPU OS 17_1 like Mac OS X) ... Mobile/15E148 Safari/604.

30

ここまでの流れ

- **道具→材料** へ：今回は Web から材料を仕入れた
- **requests** で **アクセス**、**.text** / **.json()** / **.headers** で **取り出す**
- **API / エンドポイント / パラメータ** の概念
- Wikipedia Pageviews API で **実データを取得**、NumPy で **集計**

31

次回予告

- 「七夕」の閲覧数、**最大が中央値の300倍** — なにかがおかしい
- 統計量だけでは、これ以上なにも分からない
- 次回、大原則の出番：==Always plot your data first.==
 - 描けば、山の **正体** と **日付** が分かる
 - そして可視化編の冒頭で見せた **謎の図** の正体も明かされる

32

参考リンク

- [requests Documentation](#) ↗
- [Wikimedia REST API](#) ↗
- [Pageviews API ドキュメント](#) ↗
- [HTTPステータスコード一覧 \(MDN\)](#) ↗

33